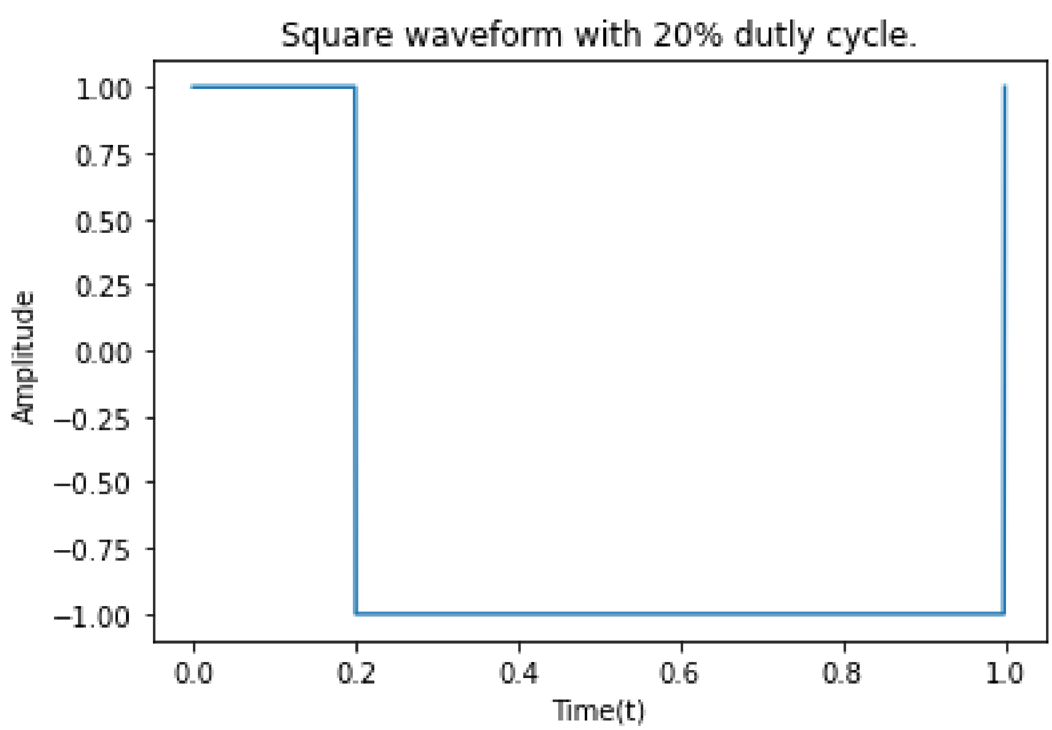
Basics of Signals and Systems - PartII

Experiment 2.1: Signal Generation: Square wave with a different duty cycle (USE scipy Library)

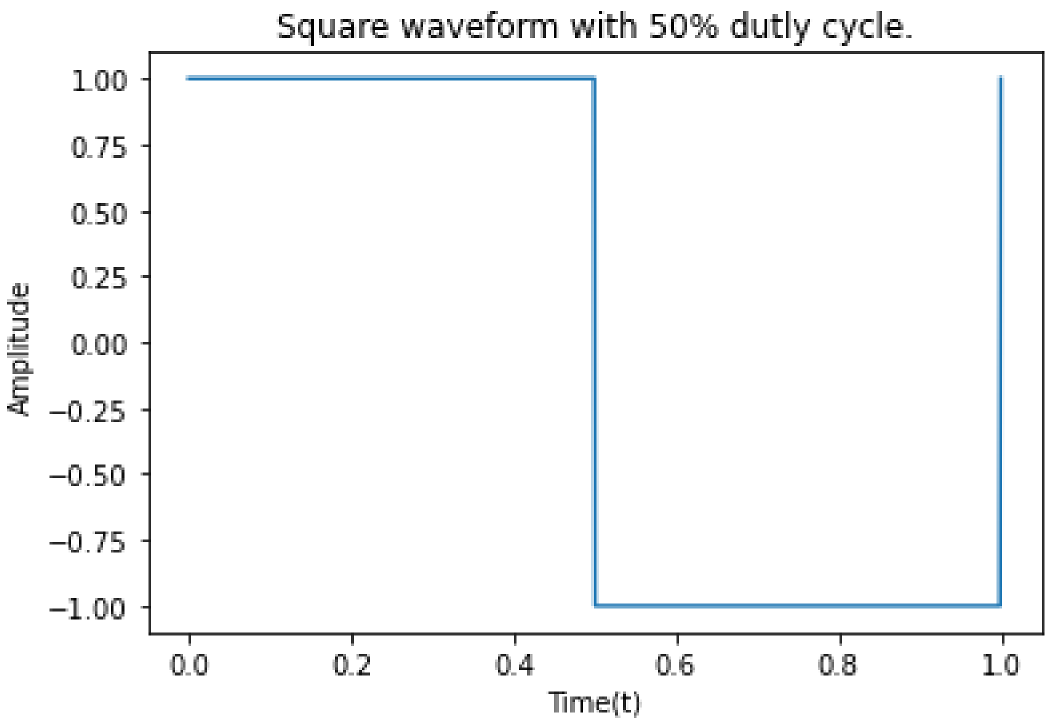
```
import matplotlib.pyplot as plt
from scipy import signal
import numpy as np
t = np.linspace(0,1,1000)

#duty cycles:-
d1 = 0.2
d2 = 0.5
d3 = 0.8

y1 = signal.square(2*np.pi*1*t,d1)
y2 = signal.square(2*np.pi*1*t,d2)
y3 = signal.square(2*np.pi*1*t,d3)

plt.plot(t,y1)
plt.xlabel("Time(t)")
plt.ylabel("Amplitude")
plt.title("Square waveform with 20% dutly cycle.")

Text(0.5, 1.0, 'Square waveform with 20% dutly cycle.')
```




```
plt.plot(t,y2)
plt.xlabel("Time(t)")
```

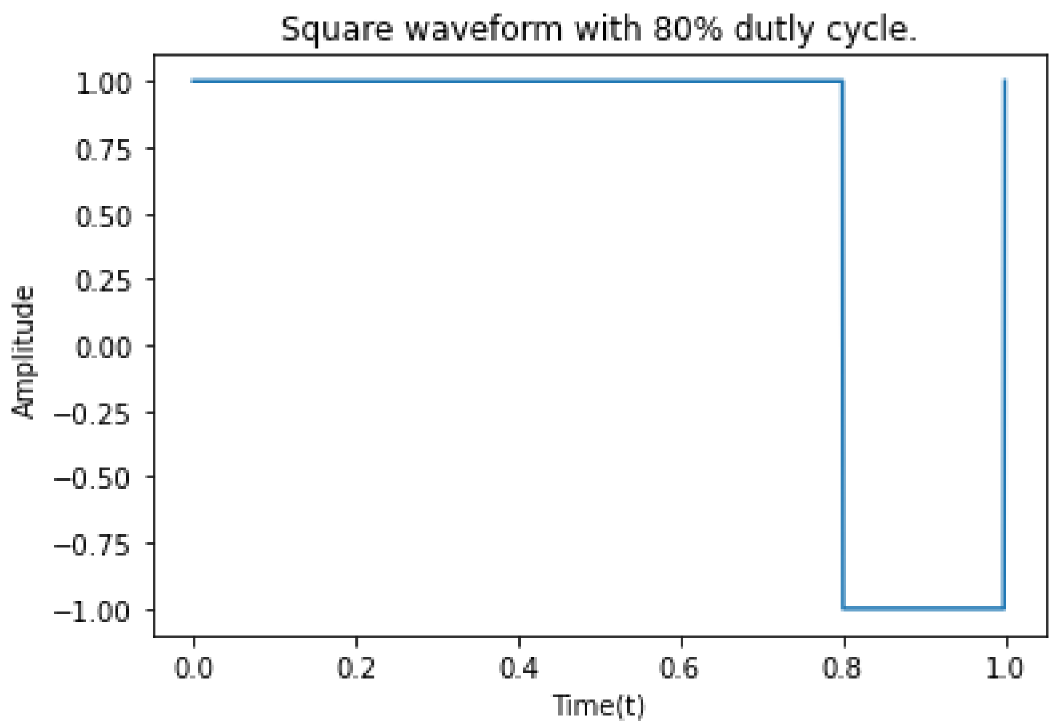
```
plt.ylabel("Amplitude")
plt.title("Square waveform with 50% dutly cycle.")
```

```
Text(0.5, 1.0, 'Square waveform with 50% dutly cycle.')
```




```
plt.plot(t,y3)
plt.xlabel("Time(t)")
plt.ylabel("Amplitude")
plt.title("Square waveform with 80% dutly cycle.")
```

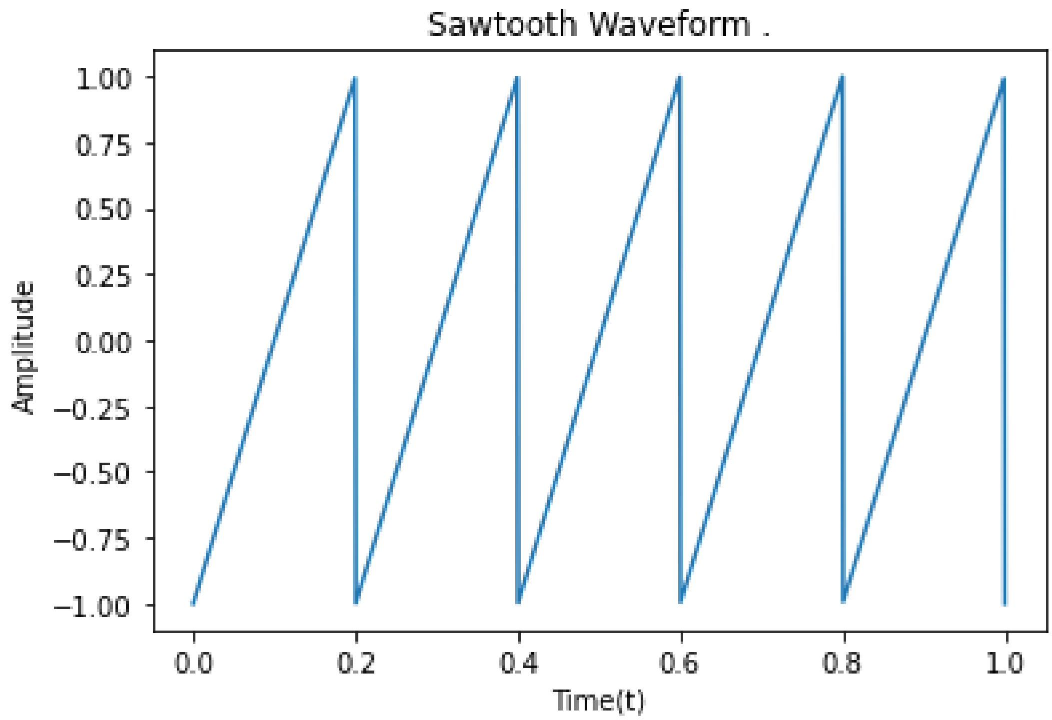
```
Text(0.5, 1.0, 'Square waveform with 80% dutly cycle.')
```



Experiment 2.2:Triangle and Sawtooth Waveform (USE scipy Library)

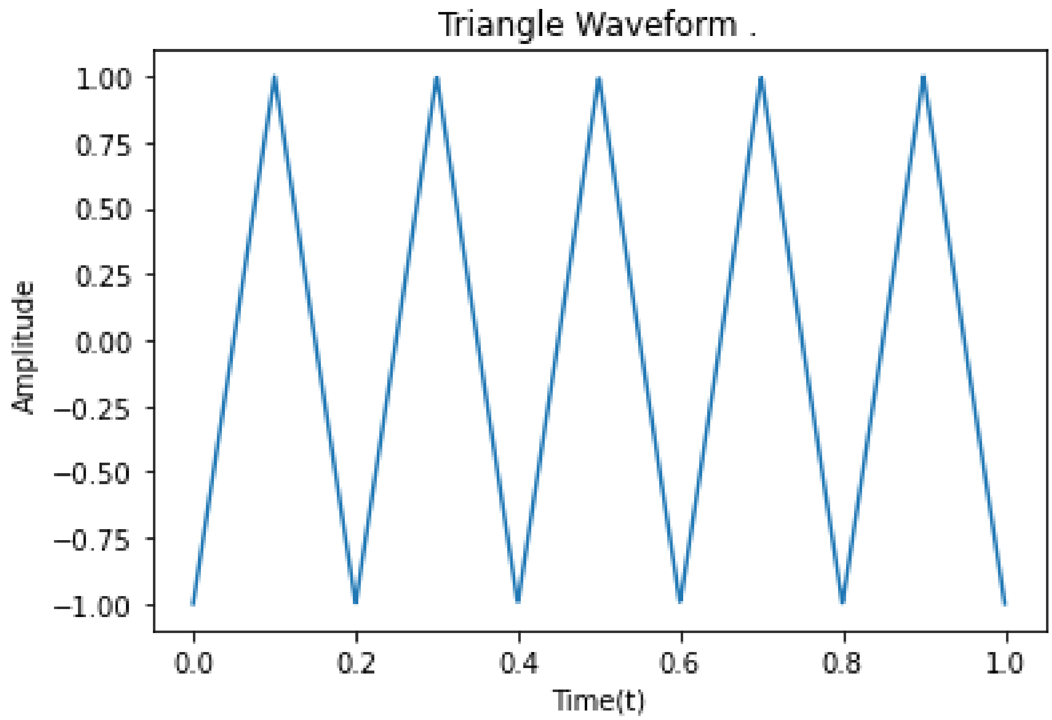
```
import matplotlib.pyplot as plt
from scipy import signal
import numpy as np

t = np.linspace(0,1,1000)

y4 = signal.sawtooth(2*np.pi*5*t)
y5 = signal.sawtooth(2*np.pi*5*t,0.5)

plt.plot(t,y4)
plt.xlabel("Time(t)")
plt.ylabel("Amplitude")
plt.title("Sawtooth Waveform .")
```

```
Text(0.5, 1.0, 'Sawtooth Waveform .')
```




```
plt.plot(t,y5)
plt.xlabel("Time(t)")
plt.ylabel("Amplitude")
plt.title("Triangle Waveform .")
```

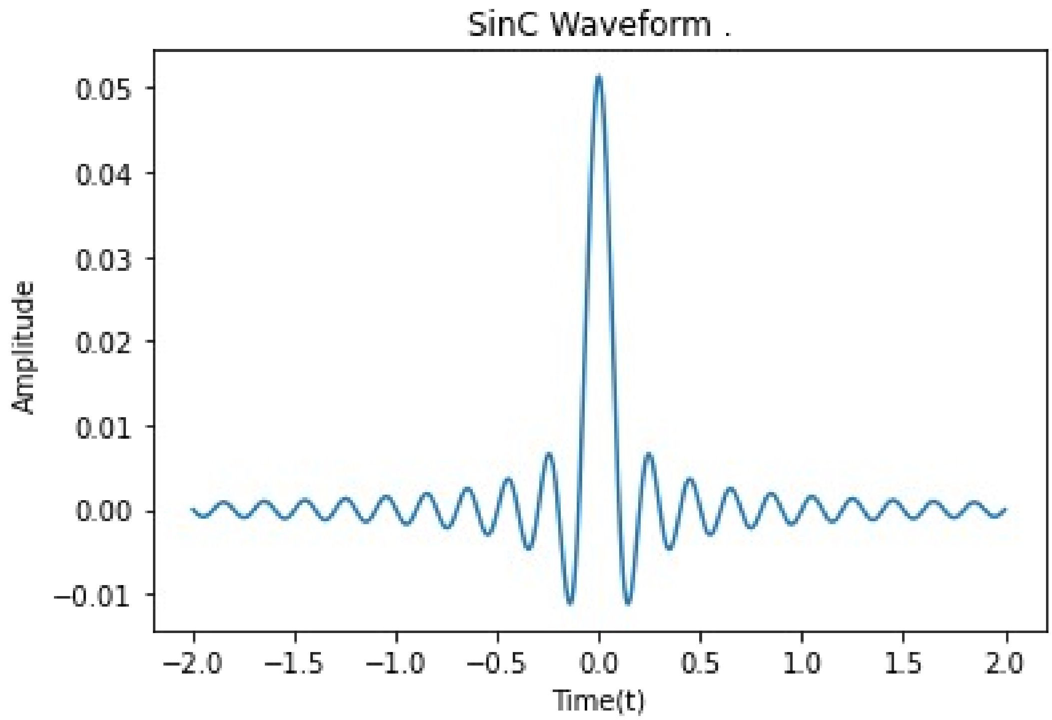
```
Text(0.5, 1.0, 'Triangle Waveform .')
```



Experiment 2.3:Generation of sinc function (USE Numpy)

```
import matplotlib.pyplot as plt
from scipy import signal
import numpy as np

t = np.linspace(-2,2,1000)

y6 = (np.sin(2*np.pi*5*t))/(2*np.pi**5*t)

plt.plot(t,y6)
plt.xlabel("Time(t)")
plt.ylabel("Amplitude")
plt.title("SinC Waveform .")
```

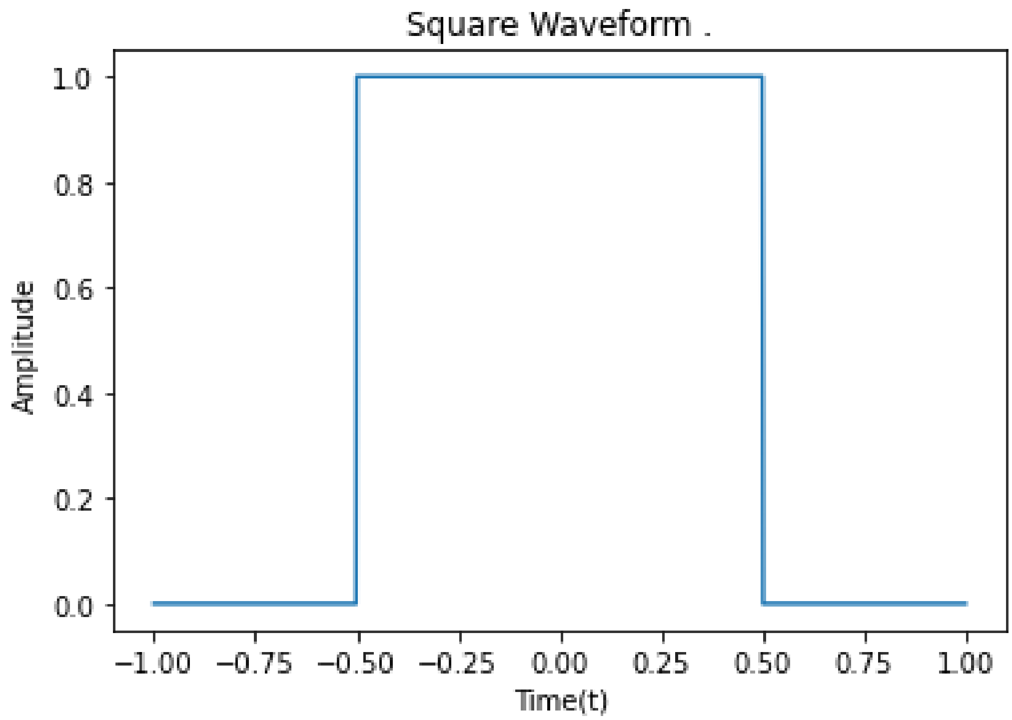
```
Text(0.5, 1.0, 'SinC Waveform .')
```



Experiment 2.4:Generation of rectangular and triangular pulse signal (USE Numpy)

```
import matplotlib.pyplot as plt
from scipy import signal
import numpy as np

t = np.linspace(-1,1,1000)

y7 = np.where(np.abs(t)<= 0.5,1,0)
y10 =t*y7
plt.plot(t,y7)
plt.xlabel("Time(t)")
plt.ylabel("Amplitude")
plt.title(" Waveform .")
```

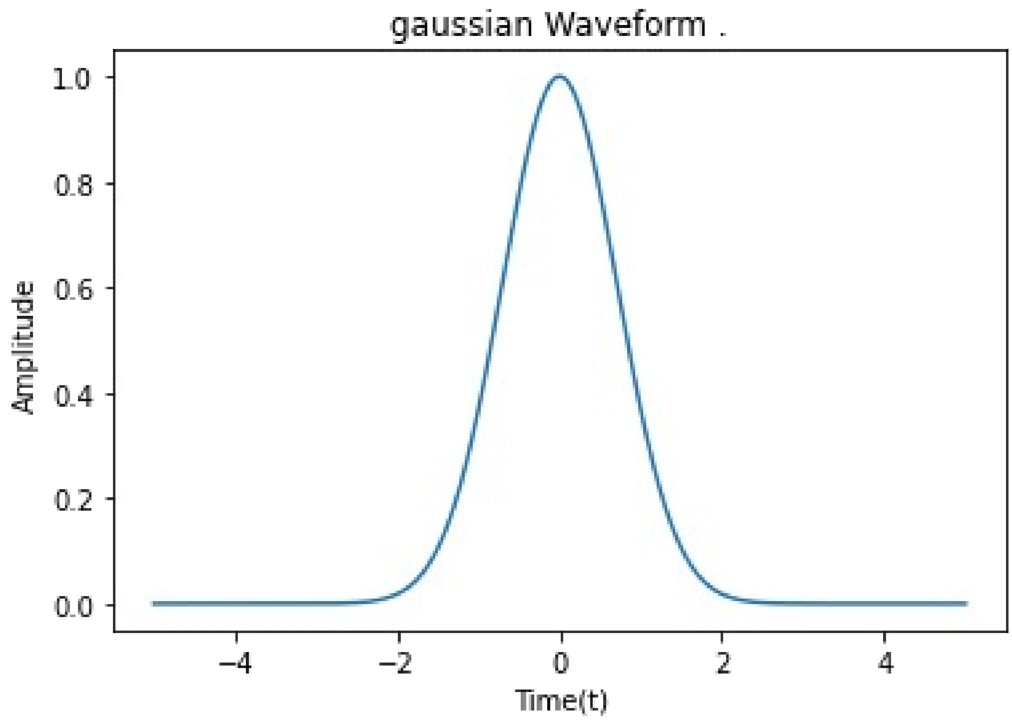
```
Text(0.5, 1.0, 'Square Waveform .')
```




```
plt.plot(t,y7)
plt.xlabel("Time(t)")
plt.ylabel("Amplitude")
plt.title(" Waveform .")
```

Experiment 2.5:Generation of Gaussian function (USE Numpy)

```
import matplotlib.pyplot as plt
from scipy import signal
import numpy as np

t = np.linspace(-5,5,1000)

y8 = (np.exp(-((t-1)/2)**2))/(2*(2**0.5))
y9 = np.exp(-(t**2))

plt.plot(t,y9)
plt.xlabel("Time(t)")
plt.ylabel("Amplitude")
plt.title("gaussian Waveform .")

Text(0.5, 1.0, 'gaussian Waveform .')
```

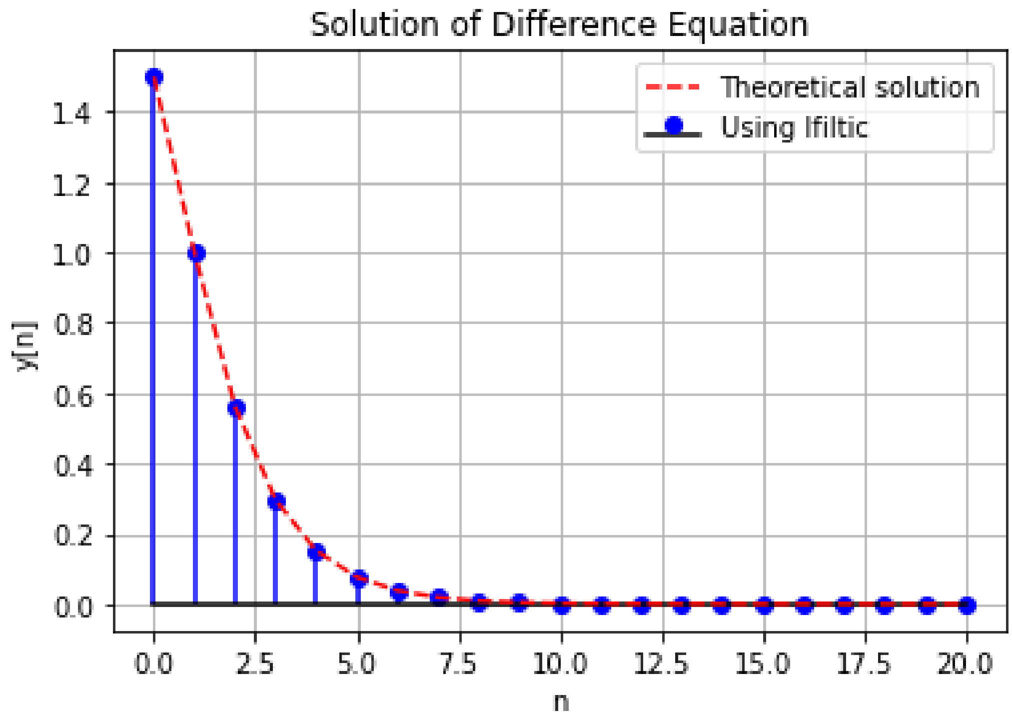


Experiment 2.6: Obtain the total solution of the difference equation y[n]-0.5y[n-1]=x[n] if the input is x[n] =(0.25)^n u[n] and the intitial condition is y[-1]=1. Use signal.lfiltic function

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import lfiltic, lfilter

# Time index
n = np.arange(0, 21)

# Input signal x[n] = (0.25)^n u[n]
x = (0.25) ** n

# Difference equation:
# y[n] - 0.5*y[n-1] = x[n]
# => y[n] = 0.5*y[n-1] + x[n]

b = [1]
a = [1, -0.5]

# Initial condition: y[-1] = 1
zi = lfiltic(b, a, y=[1])

# Calculate output
y, zf = lfilter(b, a, x, zi=zi)

# Theoretical total solution
y_theory = 2.5 * (0.5 ** n) - (0.25 ** n)
```

```python
# Display results
print(" n       x[n]       y[n]")
for i in range(len(n)):
    print(f"{n[i]:2d}    {x[i]:8.5f}    {y[i]:8.5f}")

# Plot
plt.stem(n, y, linefmt='b-', markerfmt='bo', basefmt='k-',
         label='Using lfiltic')
plt.plot(n, y_theory, 'r--', label='Theoretical solution')

plt.xlabel('n')
plt.ylabel('y[n]')
plt.title('Solution of Difference Equation')
plt.grid(True)
plt.legend()
plt.show()
```

```
 n       x[n]       y[n]
 0     1.00000     1.50000
 1     0.25000     1.00000
 2     0.06250     0.56250
 3     0.01562     0.29688
 4     0.00391     0.15234
 5     0.00098     0.07715
 6     0.00024     0.03882
 7     0.00006     0.01947
 8     0.00002     0.00975
 9     0.00000     0.00488
10     0.00000     0.00244
11     0.00000     0.00122
12     0.00000     0.00061
13     0.00000     0.00031
14     0.00000     0.00015
15     0.00000     0.00008
16     0.00000     0.00004
17     0.00000     0.00002
18     0.00000     0.00001
19     0.00000     0.00000
20     0.00000     0.00000
```



Solution of Difference Equation
Theoretical solution
Using lfiltic
y[n]
n
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0